22

DEEP DIVE 22 · అవసరమైనప్పుడే కనిపించని LOGS

Design a Logging Framework

> Log4j · SLF4J · Winston · Zap — ప్రతి system lo ఉండేది, మరియు దాదాపు ఎవరూ దీని design గురించి ఆలోచించనిది.

మూడు విరుపులు. మొదటిది **ఆపేసిన logs కోసం** 2 లక్షల సార్లు పని చేస్తుంది. రెండోది request p99 ని **1 ms నుంచి 21 ms** చేస్తుంది. మూడోది — ఒక ఘటన సమయంలో — **47% ERROR logs ని మింగేస్తుంది.**

తెలుగు + ENGLISH

YASWANTH · DEEP DIVE 22

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి **ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.**

> **DEEP DIVE 07 (PUB-SUB) చదివారా? – అక్కడి పాఠాలని ఇక్కడ మళ్ళీ చెప్పను**
>
> Pub-Sub lo ఇప్పటికే చూశాం: subscribers ని వేరుచేయడం, ఒక్కొక్కరికీ queue, ack/retry/DLQ, at-most-once vs at-least-once. **"Async గా చేస్తే publisher ఆగడు"** అనే పాఠం అక్కడిది.
>
> Logging lo మూడు **కొత్త** ప్రశ్నలు ఉన్నాయి, మరియు అవే ఈ doc:
>
> **1** • ఒక సందేశం **ఎప్పుడూ రాయబడదు** అని తెలిసినా, దాన్ని **తయారు చేస్తామా?** (§4)
> **2** • Logging **request thread ని ఆపగలదా?** (§7)
> **3** • Queue నిండితే – **ఏ logs వదిలేయాలి?** (§10) ← ఇదే ఈ doc యొక్క కేంద్రం

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు logging ఎప్పుడు పట్టింపు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ఆపేసిన log కూడా ఖరీదే

3. Step – స్థాయి, సందేశం, sink
4. **మొదటి విరుపు** – 0 పంక్తులు, 2,00,000 సార్లు పని
5. Step – సందేశాన్ని సోమరిగా చేయడం

Part 3 – రెండో విరుపు: request thread ఆగిపోవడం

6. Step – File కి రాయడం
7. **రెండో విరుపు** – p99 1 ms నుంచి 21 ms
8. Step – Queue · మరియు కొత్త ప్రశ్న

Part 4 – మూడో విరుపు: నిండిన queue

9. Step – Queue కి ఒక హద్దు
10. **మూడో విరుపు** – 47% ERROR logs మాయం
11. Step – స్థాయి-తెలిసిన queue

Part 5 – పూర్తి system

12. Step – Context · ఒక log పంక్తి ఎవరిది?
13. మొత్తం code · నడిపి చూద్దాం · **O(1) shedding నిర్ధారణ**

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటితో చెప్పాల్సిన English script
16. Follow-ups – sampling, rotation, distributed tracing
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

---

1. అడిగింది ఏమిటి – మరియు logging ఎప్పుడు పట్టింపు

> *"Design a logging framework. It should support log levels, multiple output destinations, and be usable from anywhere in the application."*

ఇది **సులభంగా అనిపిస్తుంది** – ఒక `if` (స్థాయి తనిఖీ) మరియు ఒక `console.log`. పది పంక్తులు.

LOGGING యొక్క విచిత్రమైన లక్షణం

ఒక feature – దాని విలువ సరిగ్గా దాని ఖర్చు ఎక్కువైనప్పుడే పెరుగుతుంది

సాధారణ రోజ

కొన్ని వేల logs · ఎవరూ చదవరు

ఖర్చు తక్కువ · విలువ తక్కువ
– ఇక్కడ ఏ design ఐనా పనిచేస్తుంది

ఒక ఘటన (incident)

logs **వందల రెట్లు** · అందరూ చూస్తారు

ఖర్చు గరిష్ఠం · విలువ గరిష్ఠం
– ఇక్కడే ప్రతి design విరుగుతుంది

అందుకే ఇది ఒక మంచి interview problem

సర్వర్ కష్టపడుతున్నప్పుడు – CPU నిండి, disk నెమ్మదై, errors పెరిగినప్పుడు –

logging system **అత్యధిక భారం** మోస్తుంది, మరియు **అత్యధికంగా కావాల్సింది** అప్పుడే.

"సాధారణ సమయంలో పనిచేస్తుంది" అనేది ఒక logging system కి **ఏ హామీ కాదు**.

ఈ doc lo మూడు విరుపులూ సరిగ్గా ఆ క్షణంలోనే కనిపిస్తాయి.

---

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
సెకనుకి ఎన్ని logs? ఘటన సమయంలో ఎన్ని?	§10 – queue పరిమాణం మరియు drop విధానం
Log పంక్తి ఎక్కడికి – console, file, network?	Network అంటే **ఆలస్యం ఖాయం** (§7)

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Logging request ని ఆపొచ్చా?	**దాదాపు ఎప్పుడూ కాదు** – §8
Logs పోతే పర్వాలేదా?	**ఏవి పోవచ్చో** అడగండి – §10
Structured (JSON) నా, plain text నా?	§12 – fields ఉంటే వెతకడం సులభం
స్థాయిని నడుస్తుండగా మార్చగలమా?	ఒక `setLevel` – కానీ §5 ఆ తనిఖీని చౌకగా ఉంచాలి
ఒక request యొక్క అన్ని logs ని కలిపి చూడగలమా?	§12 – context / child loggers

నాలుగో ప్రశ్న – మరియు ఇది INTERVIEW ని మలుపు తిప్పుతుంది

"Logs కొన్ని పోతే పర్వాలేదా?"

చాలా మంది "అవును, logs అంత ముఖ్యం కాదు" అంటారు. అది **సగం జవాబు**.

సరైన ప్రశ్న: **"ఏవి పోవచ్చు?"** – DEBUG logs లక్షలు పోయినా ఎవరూ గమనించరు. కానీ ఒక్క ERROR పోతే, **మీరు వెతుకుతున్న సమాధానమే పోయినట్టు**.

మరియు §10 lo చూస్తాం: **సహజమైన queue design సరిగ్గా అదే చేస్తుంది** – ఘటన సమయంలో 47% ERROR logs ని మింగేస్తుంది.

Part 2 – మొదటి విరుపు: ఆపేసిన log కూడా ఖరీదే

---

3. Step – స్థాయి, సందేశం, sink

```
const LEVELS = { DEBUG: 10, INFO: 20, WARN: 30, ERROR: 40 };
class Logger {
  constructor(level = 'INFO', sink = console.log) {
    this.level = LEVELS[level]; this.sink = sink;
  }
  #log(lvl, msg) {
    if (LEVELS[lvl] < this.level) return;              // స్థాయి తనిఖీ
    this.sink(`[${lvl}] ${msg}`);
  }
  debug(m) { this.#log('DEBUG', m); }
  info(m)  { this.#log('INFO',  m); }
  warn(m)  { this.#log('WARN',  m); }
  error(m) { this.#log('ERROR', m); }
}
```

పనిచేస్తుంది, మరియు ఇది ప్రతి tutorial lo ఉండే code:

```
[INFO] server మొదలైంది
[ERROR] DB కనెక్షన్ విఫలం
```

`DEBUG` పంక్తి కనిపించలేదు – స్థాయి `INFO` కాబట్టి. **సరిగ్గా ఉంది.**

---

4. మొదటి విరుపు – 0 పంక్తులు, 2,00,000 సార్లు పని

ఇప్పుడు ఒక సాధారణ వాడకం చూద్దాం – DEBUG lo ఒక వివరమైన సందేశం:

```
log.debug('request: ' + describeRequest(req));
```

స్థాయి `INFO`, కాబట్టి ఇది **ఎప్పుడూ రాయబడదు**. 2,00,000 సార్లు పిలుద్దాం:

```
  2,00,000 log.debug() calls · స్థాయి INFO (DEBUG ఆఫ్)
  బయటికి వచ్చిన పంక్తులు : 0
  describeRequest() calls: 2,00,000
  సమయం                  : 69 ms
```

మొదటి విరుపు – స్థాయి తనిఖీ చాలా ఆలస్యంగా జరుగుతోంది

ఒక్క పంక్తి కూడా రాయలేదు. అయినా `describeRequest()` **2,00,000 సార్లు** నడిచింది, మరియు 69 ms వృథా అయింది.

కారణం JavaScript కాదు – ఇది ప్రతి భాషలోనూ ఇలాగే. `log.debug(x)` అనేది ఒక **function call**. Function కి వెళ్ళే ముందు దాని argument తయారుకావాలి. అంటే string కలపడం, `JSON.stringify`, అంతా – **logger ని చేరకముందే**.

Logger lo ఉన్న `if` ఆ పని అయిపోయాక నడుస్తుంది. అది **రాయడాన్ని** ఆపుతుంది, **తయారీని** కాదు.

మరియు ఇది ఎక్కడ కరుస్తుంది? – DEBUG logs ఎక్కువగా ఉండేది **వేడిగా నడిచే code lo**. అంటే ఈ ఖర్చు సరిగ్గా అత్యంత ముఖ్యమైన చోట పడుతుంది, మరియు **production lo DEBUG ఆఫ్ చేసినా అది తగ్గదు**.

---

5. Step – సందేశాన్ని సోమరిగా వేయడం

రెండు పరిష్కారాలు, రెండూ ఒకే ఆలోచన: **సందేశాన్ని తయారుచేయడాన్ని స్థాయి తనిఖీ తర్వాతికి వాయిదా వేయడం.**

```
isEnabled(l) { return LEVELS[l] >= this.level; }

#log(l, m) {
  if (LEVELS[l] < this.level) return;
  // సందేశం ఒక function అయితే – ఇప్పుడే దాన్ని పిలవడం
  this.sink(`[${l}] ${typeof m === 'function' ? m() : m}`);
}
```

```
2,00,000 log.debug() calls · స్థాయి INFO (DEBUG ఆఫ్)

  string కలిపి (§4)     →   70 ms · describeRequest() 2,00,000 సార్లు
  function గా ఇస్తే     →    2 ms · describeRequest() 0 సార్లు
  isEnabled తనిఖీతో     →    1 ms · describeRequest() 0 సార్లు

మరియు DEBUG ఆన్ చేస్తే – మూడూ ఒకే పని చేయాలి:
  string కలిపి    → ["[DEBUG] x"]
  function గా     → ["[DEBUG] x"]
```

70 ms → 2 ms, మరియు ఖరీదైన function **సున్నా సార్లు**.

సమయాలు runs మధ్య కొంచెం ఊగుతాయి – మొదటి వరుస 67-70 ms, మిగతా రెండూ 1-2 ms. కానీ **calls నిలువు వరుస ఖచ్చితమైనది**: 2,00,000 మరియు 0, ప్రతి run lo. Deep Dive 13 §7 lo చెప్పిన అదే విషయం – *గడియారం ఊగితే, పనిని లెక్కించండి.*

రెండు పరిష్కారాలూ ఎందుకు ఉన్నాయి

`log.debug(() => ...)` – **సొగసైనది**. ఒక closure మాత్రమే అదనపు ఖర్చు, మరియు call site శుభ్రంగా ఉంటుంది.

`if (log.isEnabled('DEBUG')) log.debug(...)` – **మరింత చౌకైనది** (closure కూడా లేదు), కానీ call site lo రెండు పంక్తులు. పాత Java code lo `if (logger.isDebugEnabled())` ఎక్కడ చూసినా ఇదే కారణం.

మూడో మార్గం కూడా ఉంది: చాలా ఆధునిక libraries (Go యొక్క `zap`, Rust యొక్క `tracing`) సందేశాన్ని **ముక్కలుగా** తీసుకుంటాయి – `log.debug("request", { id, path })` – మరియు ఆ ముక్కలని **కలపడం** స్థాయి తనిఖీ తర్వాతే చేస్తాయి. §13 lo `fields` అదే.

Part 3 – రెండో విరుపు: request thread ఆగిపోవడం

6. Step – File కి రాయడం

Console సరిపోదు; logs ఒక file కి వెళ్ళాలి:

```
class SyncFileLogger {
  constructor(file) { this.fd = fs.openSync(file, 'a'); }
  log(line) { fs.writeSync(this.fd, line + '\n'); }
}
```

సూటిగా ఉంది, మరియు నమ్మదగినది – ప్రతి పంక్తి వెంటనే disk కి వెళుతుంది, crash అయినా పోదు. కొలిచి చూద్దాం:

```
50,000 log పంక్తులు · ఒక్కొక్కటి 66 bytes

  విధానం             |   మొత్తం  |  ఒక్కో పంక్తికి  |  write() calls
  -------------------+-----------+------------------+----------------
  ప్రతిదీ నేరుగా     |     77 ms |           1.5 µs |         50,000
  500 గుంపుగా        |      2 ms |           0.0 µs |            100
```

ఒక్కో పంక్తికి **1.5 µs** – అది భయంకరమైనది కాదు. గుంపుగా రాస్తే 38 రెట్లు వేగం, కానీ మొత్తం 77 ms అనేది భరించదగినదే.

అయితే సమస్య ఏమిటి? – ఆ **సగటు** ఒక అబద్ధం.

7. రెండో విరుపు – p99 1 ms నుంచి 21 ms

నిజమైన appenders **ఏకరీతిగా ఉండవు**. Disk buffer నిండినప్పుడు, log file rotate అయినప్పుడు, network appender ఒక packet కోల్పోయినప్పుడు – ఒక్క call **మిల్లీసెకన్లు** ఆగుతుంది.

ప్రతి 200వ పంక్తికీ 20 ms ఆగే ఒక appender, 5,000 requests, ఒక్కో request కి 10 logs:

```
  విధానం          |  సగటు   |   p50   |   p99    |  గరిష్ఠం
  ----------------+---------+---------+----------+----------
  request path lo |  2.00ms |  1.00ms |  21.00ms |   21.02ms
  queue lo        |  1.00ms |  1.00ms |   1.01ms |    1.05ms
```

ఒక APPENDER ఆగితే · ఆ ఆలస్యం ఎవరిది?

20 ms disk flush – అది ఎక్కడ కనిపిస్తుంది

Request path lo	Queue lo
ఆ 20 ms ని **ఒక customer** చూస్తాడు p99 = **21 ms** · గరిష్ఠం 21.02 ms అతను log రాయమని అడగలేదు	ఆ 20 ms **background lo** గడుస్తుంది p99 = **1.01 ms** · గరిష్ఠం 1.05 ms Request ముగిసిపోయింది

సగటు కూడా సగం అయింది (2.00 → 1.00 ms) – కానీ **సగటు ముఖ్యం కాదు.**
ముఖ్యమైనది p99: ప్రతి వందలో ఒక customer **21 రెట్లు** ఎక్కువ ఆగాడు –
మరియు ఆ ఆలస్యానికి అతని request తో **ఎలాంటి సంబంధమూ లేదు.**

రెండో విరుపు – LOGGING ఒక CUSTOMER యొక్క సమయాన్ని తింటోంది

Customer ఒక order ఇచ్చాడు. అతని request 1 ms పని చేసి, ఆపై **20 ms disk flush కోసం ఆగింది** – అతను అడగని, అతనికి ప్రయోజనం లేని పని.

మరియు ఇది **సరిగ్గా అత్యంత చెడ్డ సమయంలో** ఘోరమవుతుంది: సర్వర్ కష్టపడుతున్నప్పుడు disk నెమ్మదవుతుంది, logs పెరుగుతాయి, మరియు ప్రతి request మరింత ఆగుతుంది. **Logging ఒక feedback loop అవుతుంది.**

మౌలిక తప్పు: "Log రాయడం" అనేది రెండు పనులు – **ఏమి రాయాలో నిర్ణయించడం** (వేగం, request కి సంబంధించినది) మరియు **దాన్ని ఎక్కడికో పంపడం** (నెమ్మది, request కి సంబంధం లేనిది). వాటిని ఒకే call lo కలపడం వల్ల రెండోది మొదటిదాని మీద పడుతుంది.

---

8. Step – Queue · మరియు కొత్త ప్రశ్న

పరిష్కారం స్పష్టం: **record ని ఒక queue lo వేసి, request ని వదిలేయడం.** వేరే ఒక చోట (thread, timer, event-loop tick) దాన్ని appenders కి పంపడం.

```
#emit(level, msg, fields) {
  if (LEVELS[level] < this.level) return { ok: true, skipped: true };
  const text = typeof msg === 'function' ? msg() : msg;
  const rec = new Record(level, text, { ...this.fields, ...fields }, this.clock());
  // ← request ఇక్కడే ముగుస్తుంది
  if (this.queue) return this.queue.push(rec);
  for (const a of this.appenders) a.write([rec]);
  return { ok: true };
}
```

Deep Dive 07 §7 lo సరిగ్గా ఇదే ఆలోచన వాడాం – slow subscriber publisher ని ఆపకుండా. **ఇక్కడ కొత్తది ఆ queue కాదు; దాని తర్వాత వచ్చే ప్రశ్న.**
మరియు ఆ ప్రశ్న ఇది: **queue ఎంత పెద్దది?**

- **అపరిమితం** అంటే – producer consumer కంటే వేగంగా ఉంటే queue పెరుగుతూనే ఉంటుంది. ఒక ఘటన సమయంలో logs వందల రెట్లు పెరిగితే, **logging system ఒక memory leak అవుతుంది**

మరియు process చనిపోతుంది. *Logs కోసం సర్వర్ చనిపోవడం* – అది అసలు ఉద్దేశానికి పూర్తి వ్యతిరేకం.

- **పరిమితం** అంటే – నిండినప్పుడు **ఏదో ఒకటి వదిలేయాలి.**

ఏది వదిలేయాలి? అదే §10.

Part 4 – మూడో విరుపు: నిండిన queue

9. Step – Queue కి ఒక హద్దు

సహజమైన రెండు ఎంపికలు:

```
if (q.length < capacity) q.push(rec);
else if (policy === 'DROP_NEW') dropped++;              // కొత్తదాన్ని వదిలేయడం
else if (policy === 'DROP_OLD') { q.shift(); q.push(rec); }   // పాతదాన్ని వదిలేయడం
```

రెండూ సహేతుకంగా అనిపిస్తాయి, మరియు రెండూ నిజమైన libraries lo ఉన్నాయి. **DROP_NEW** ఇప్పటికే ఉన్న చరిత్రని కాపాడుతుంది; **DROP_OLD** తాజా స్థితిని కాపాడుతుంది.

ఇప్పుడు **ఒక ఘటనని అనుకరిద్దాం** – 2,00,000 log events, ఘటన సమయపు నిష్పత్తులతో (DEBUG 50%, INFO 20%, WARN 15%, ERROR 15%), queue 5,000, consumer producer కంటే నెమ్మది:

10. మూడో విరుపు – 47% ERROR logs మాయం

```
2,00,000 log events (ఘటన సమయం) · queue 5,000 · consumer నెమ్మది

  మొత్తం: DEBUG 1,00,084 · INFO 39,911 · WARN 30,178 · ERROR 29,827

  విధానం        |  ERROR రాసినవి |  ERROR పోయినవి |  ERROR నిలిచిన శాతం
  --------------+----------------+----------------+---------------------
  DROP_NEW      |         15,785 |         14,042 |                52.9%
  DROP_OLD      |         15,778 |         14,049 |                52.9%
  DROP_LOWEST   |         29,827 |              0 |               100.0%
```

మూడో విరుపు – మీరు వెతుకుతున్న LOGS సరిగ్గా అవే పోయాయి

DROP_NEW మరియు DROP_OLD – రెండూ 47% ERROR logs ని మింగేశాయి. మరియు అవి ఒకేలా చెడ్డవి కావడం యాదృచ్ఛికం కాదు: **రెండూ వచ్చిన క్రమాన్ని బట్టి వదిలేస్తాయి**, ఏమి ఉందో చూడకుండా.

Queue lo 70% DEBUG ఉంది. కాబట్టి "ఏదో ఒకటి వదిలేయి" అంటే – గణితం ప్రకారం, **అదే నిష్పత్తిలో ERROR లు కూడా పోతాయి**.

మరియు ఇది ఎప్పుడు జరిగింది? – సరిగ్గా ఘటన సమయంలో. సాధారణ రోజున queue ఎప్పుడూ నిండదు, కాబట్టి ఏమీ పోదు. Queue నిండేది system కష్టపడుతున్నప్పుడే – అంటే మీరు ఆ ERROR logs కోసం **వెతుకుతున్న ఆ క్షణంలోనే**.

మౌలిక తప్పు: Queue ఒక **FIFO** అని అనుకున్నాం. కానీ log records **సమానమైనవి కావు** – ఒక ERROR మరియు ఒక DEBUG కి విలువలో వందల రెట్లు తేడా ఉంది. సమాన వస్తువుల కోసం రాసిన ఒక structure ని అసమాన వస్తువులకి వాడాం.

---

11. Step – స్థాయి-తెలిసిన queue

పరిష్కారం: **నిండినప్పుడు, అత్యంత తక్కువ విలువైన దాన్ని వదిలేయడం** – వచ్చిన క్రమాన్ని కాదు.

సరళమైన అమలు queue అంతటినీ వెతుకుతుంది – O(n) ప్రతి drop కీ. బదులుగా **ఒక్కో స్థాయికి ఒక్కో queue** ఉంచితే అది **O(1)**:

```
// ఒక్కో స్థాయికి ఒక్కో queue – కాబట్టి "అతి తక్కువ స్థాయిని వదిలేయడం" O(1)
class LevelQueue {
  #q = new Map(ORDER.map(l => [l, []]));
  #size = 0;
  dropped = Object.fromEntries(ORDER.map(l => [l, 0]));
  constructor(capacity = 10_000) { this.capacity = capacity; }

  push(rec) {
    if (this.#size < this.capacity) {
      this.#q.get(rec.level).push(rec); this.#size++;
      return { ok: true };
    }
    // నిండిపోయింది – అత్యంత తక్కువ స్థాయి ఉన్న దాన్ని వదిలేయడం
    for (const l of ORDER) {
      if (LEVELS[l] >= LEVELS[rec.level]) break;      // కొత్తదే అతి తక్కువ
      const q = this.#q.get(l);
      if (q.length) {
        q.shift(); this.dropped[l]++;
        this.#q.get(rec.level).push(rec);
        return { ok: true, shed: l };
      }
    }
    this.dropped[rec.level]++;
    return { ok: false, reason: 'QUEUE_FULL' };
  }
  // ఎక్కువ స్థాయి ముందు బయటికి
  drain(n) {
```

```
    const out = [];
    for (let i = ORDER.length - 1; i >= 0 && out.length < n; i--) {
      const q = this.#q.get(ORDER[i]);
      while (q.length && out.length < n) { out.push(q.shift()); this.#size--; }
    }
    return out;
  }
}
```

మూడు వివరాలు – ప్రతిదీ ఉద్దేశపూర్వకం

1 · `if (LEVELS[l] >= LEVELS[rec.level]) break;` – కొత్తదే అతి తక్కువ స్థాయి అయితే, **దాన్నే వదిలేయాలి**. ఒక DEBUG కోసం ఒక INFO ని బలి ఇవ్వకూడదు.

2 · `drain` **ఎక్కువ స్థాయి ముందు ఇస్తుంది** – consumer నెమ్మదిగా ఉంటే, బయటికి వెళ్ళే కొద్దిపాటి logs **ముఖ్యమైనవే** కావాలి.

3 · `dropped` **ఒక్కో స్థాయికీ లెక్కిస్తుంది** – ఇది కేవలం గణాంకం కాదు. "గత గంటలో 4 లక్షల DEBUG వదిలేశాం" అనేది ఒక **ఆరోగ్య సంకేతం**; "3 ERROR వదిలేశాం" అనేది ఒక **ప్రమాద ఘంటిక**. రెండూ ఒకేలా చూపించకూడదు.

రెండు అమలులూ ఒకే ఫలితం ఇస్తాయా?

O(1) రూపం ఒక optimisation. అది O(n) రూపంతో **అక్షరాలా ఏకీభవిస్తుందా**? – 2,00,000 events మీద పోల్చి చూద్దాం:

```
స్థాయి |  LevelQueue రాసినవి |  O(n) reference |  ఏకీభవించాయా
-------+---------------------+-----------------+---------------
DEBUG  |               5,079 |           5,079 |              ✓
INFO   |              39,911 |          39,911 |              ✓
WARN   |              30,178 |          30,178 |              ✓
ERROR  |              29,827 |          29,827 |              ✓

ERROR నిలిచిన శాతం : 100.0%
మొత్తం సమయం        : 34 ms  (O(1) shedding)
```

నాలుగు స్థాయిలూ సరిగ్గా సరిపోయాయి. మరియు ఆ DEBUG వరుస చూడండి – 1,00,084 lo **5,079 మాత్రమే** రాయబడ్డాయి. **95% DEBUG పోయింది, 0% ERROR పోయింది.** అదే సరైన వ్యాపారం.

Part 5 – పూర్తి system

12. Step – Context · ఒక log పంక్తి ఎవరిది?

ఒక చివరి విషయం, మరియు ఇది ఆచరణలో చాలా ముఖ్యమైనది.

సర్వర్ lo వంద requests ఒకేసారి నడుస్తున్నాయి. Log file lo వాటి పంక్తులు **కలిసిపోయి** ఉంటాయి. ఒక customer ఫిర్యాదు చేస్తే – **అతని request యొక్క పంక్తులు ఏవి?**

```
// ఒక context తో కొత్త logger — అదే appenders, అదే queue
child(fields) {
  const c = new Logger({ level: this.levelName, appenders: this.appenders,
                         queue: this.queue, clock: this.clock,
                         fields: { ...this.fields, ...fields } });
  return c;
}
```

```
  2026-09-23T10:00:00.000Z [INFO] order అందింది svc=orders reqId=r-42 user=ravi
items=3
  2026-09-23T10:00:00.000Z [ERROR] చెల్లింపు విఫలం svc=orders reqId=r-42 user=ravi
```

ఇది ఒక సౌకర్యం కాదు – ఇది LOGS ని వెతకగలిగేలా చేసేది

`root.child({ reqId })` అని ఒకసారి చేస్తే, ఆ request lo ప్రతి పంక్తీ **ఆ reqId మోస్తుంది** – ప్రతి `log.info()` call lo దాన్ని రాయనవసరం లేదు.

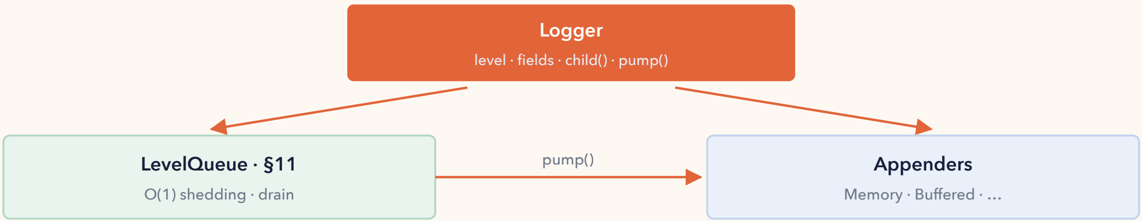
ఆపై `grep reqId=r-42` – **ఆ ఒక్క customer యొక్క పూర్తి కథ**, వంద ఇతర requests మధ్య నుంచి.

మరియు `fields` ఒక వస్తువు కావడం వల్ల – string కాకుండా – దాన్ని JSON గా కూడా రాయొచ్చు, మరియు అప్పుడు `reqId` ఒక **వెతకగలిగే క్షేత్రం** అవుతుంది, ఒక string ముక్క కాదు. **Structured logging అంటే అదే.**

13. మొత్తం code · నడిపి చూద్దాం · O(1) shedding నిర్ధారణ

నిర్మాణం · నాలుగు ముక్కలు

Logger ఏమి రాయాలో నిర్ణయిస్తుంది · appenders ఎక్కడికో పంపుతాయి



Request thread **queue.push()** వరకే వెళుతుంది – అది ఒక array push.

pump() ని వేరే చోట పిలవాలి: ఒక timer, ఒక thread, లేదా ఒక event-loop tick.

```
'use strict';
const LEVELS = { DEBUG: 10, INFO: 20, WARN: 30, ERROR: 40 };
const ORDER = ['DEBUG', 'INFO', 'WARN', 'ERROR'];

class Record {
  constructor(level, msg, fields, at) {
    Object.assign(this, { level, msg, fields, at });
  }
  format() {
    const f = Object.entries(this.fields ?? {})
      .map(([k, v]) => `${k}=${typeof v === 'string' ? v : JSON.stringify(v)}`)
      .join(' ');
    return `${new Date(this.at).toISOString()} [${this.level}] ` +
           `${this.msg}${f ? ' ' + f : ''}`;
  }
}

class MemoryAppender {
  lines = [];
  write(records) { for (const r of records) this.lines.push(r.format()); }
  close() {}
}
class BufferedAppender {                  // గుంపుగా రాయడం · §7
  constructor(sink, batch = 500) {
    Object.assign(this, { sink, batch }); this.buf = [];
  }
  write(records) {
    for (const r of records) {
      this.buf.push(r.format());
      if (this.buf.length >= this.batch) this.flush();
    }
  }
  flush() {
    if (this.buf.length) { this.sink(this.buf.join('\n')); this.buf.length = 0; }
  }
  close() { this.flush(); }
}

// ఒక్కో స్థాయికి ఒక్కో queue — "అతి తక్కువ స్థాయిని వదిలేయడం" O(1)
class LevelQueue {
```

```
  #q = new Map(ORDER.map(l => [l, []]));
  #size = 0;
  dropped = Object.fromEntries(ORDER.map(l => [l, 0]));
  constructor(capacity = 10_000) { this.capacity = capacity; }
  get size() { return this.#size; }

  push(rec) {
    if (this.#size < this.capacity) {
      this.#q.get(rec.level).push(rec); this.#size++;
      return { ok: true };
    }
    for (const l of ORDER) {
      if (LEVELS[l] >= LEVELS[rec.level]) break;     // కొత్తదే అతి తక్కువ
      const q = this.#q.get(l);
      if (q.length) {
        q.shift(); this.dropped[l]++;
        this.#q.get(rec.level).push(rec);
        return { ok: true, shed: l };
      }
    }
    this.dropped[rec.level]++;
    return { ok: false, reason: 'QUEUE_FULL' };
  }
  drain(n) {                                         // ఎక్కువ స్థాయి ముందు
    const out = [];
    for (let i = ORDER.length - 1; i >= 0 && out.length < n; i--) {
      const q = this.#q.get(ORDER[i]);
      while (q.length && out.length < n) { out.push(q.shift()); this.#size--; }
    }
    return out;
  }
  get totalDropped() { return Object.values(this.dropped).reduce((a,b) => a+b, 0); }
}

class Logger {
  constructor({ level = 'INFO', appenders = [], queue = null,
                clock = Date.now, fields = {} } = {}) {
    Object.assign(this, { appenders, queue, clock, fields });
    this.level = LEVELS[level];
    this.levelName = level;
  }
  isEnabled(l) { return LEVELS[l] >= this.level; }
  setLevel(l) { this.level = LEVELS[l]; this.levelName = l; return this; }

  // ఒక context తో కొత్త logger — అదే appenders, అదే queue
  child(fields) {
    return new Logger({ level: this.levelName, appenders: this.appenders,
                        queue: this.queue, clock: this.clock,
                        fields: { ...this.fields, ...fields } });
  }

  #emit(level, msg, fields) {
    if (LEVELS[level] < this.level) return { ok: true, skipped: true };  // §5
    // §5 · సోమరి — స్థాయి తనిఖీ తర్వాతే సందేశం తయారవుతుంది
    const text = typeof msg === 'function' ? msg() : msg;
    const rec = new Record(level, text,
                           { ...this.fields, ...fields }, this.clock());
    if (this.queue) return this.queue.push(rec);                       // §11
    for (const a of this.appenders) a.write([rec]);
```

```
    return { ok: true };
  }
  debug(m, f) { return this.#emit('DEBUG', m, f); }
  info (m, f) { return this.#emit('INFO',  m, f); }
  warn (m, f) { return this.#emit('WARN',  m, f); }
  error(m, f) { return this.#emit('ERROR', m, f); }

  // queue నుంచి appenders కి — ఒక వేరే thread/tick lo పిలవాలి
  pump(n = 1000) {
    if (!this.queue) return { written: 0 };
    const batch = this.queue.drain(n);
    if (batch.length) for (const a of this.appenders) a.write(batch);
    return { written: batch.length, queued: this.queue.size };
  }
  flush() {
    while (this.queue && this.queue.size) this.pump();
    for (const a of this.appenders) a.flush?.();
  }
  close() { this.flush(); for (const a of this.appenders) a.close(); }
}
```

```
--- ప్రాథమిక ప్రవర్తన ---
  2026-09-23T10:00:00.000Z [INFO] server మొదలైంది port=8080
  2026-09-23T10:00:00.000Z [ERROR] DB విఫలం retries=3

--- §5: సోమరి సందేశం ---
  string కలిపితే  : expensive() 1 సారి
  function గా ఇస్తే: expensive() 0 సార్లు

--- context (child logger) ---
  2026-09-23T10:00:00.000Z [INFO] order అందింది svc=orders reqId=r-42 user=ravi
items=3
  2026-09-23T10:00:00.000Z [ERROR] చెల్లింపు విఫలం svc=orders reqId=r-42 user=ravi

--- §11: queue నిండితే ఏమి వదిలేస్తాం ---
  queue నిండింది (5 DEBUG). ఇప్పుడు:
    debug d5  → { ok: false, reason: 'QUEUE_FULL' }
    error e1  → { ok: true, shed: 'DEBUG' } ← ఒక DEBUG వదిలేసింది
    error e2  → { ok: true, shed: 'DEBUG' }
  వదిలేసినవి : { DEBUG: 3, INFO: 0, WARN: 0, ERROR: 0 }
  రాసినవి    : e1 e2 d2 d3 d4

--- level మార్చడం ---
   1 పంక్తి: 2026-09-23T10:00:00.000Z [INFO] ఇప్పుడు కనిపిస్తుంది

--- buffered appender ---
  250 పంక్తులు, flush ముందు  : write() 2 సార్లు
  close() తర్వాత            : write() 3 సార్లు · 100+100+50 పంక్తులు
```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

`debug d5 → QUEUE_FULL` – queue నిండింది మరియు d5 **అతి తక్కువ స్థాయి**, కాబట్టి దాన్నే వదిలేసింది. ఒక DEBUG కోసం ఇంకో DEBUG ని బలి ఇవ్వలేదు.

`error e1 → { ok: true, shed: 'DEBUG' }` – ERROR కి చోటు కావాలి, కాబట్టి ఒక DEBUG వెళ్ళిపోయింది. మరియు **ఏది వెళ్ళిందో తిరిగి చెప్తోంది** – ఇది monitoring కి కావాల్సిన సమాచారం.

`రాసినవి : e1 e2 d2 d3 d4` – `drain` **ERROR లని ముందు** ఇచ్చింది. Consumer వెనకబడి ఉంటే, బయటికి వెళ్ళే మొదటివి ముఖ్యమైనవే.

`write() 2 సార్లు` → **`3 సార్లు`** – 250 పంక్తులు, batch 100. మూడో batch (50 పంక్తులు) `close()` వరకు buffer lo ఉంది. `close()` **పిలవకపోతే ఆ 50 పంక్తులు ఎప్పటికీ disk చేరవు** – buffered logging యొక్క ధర.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

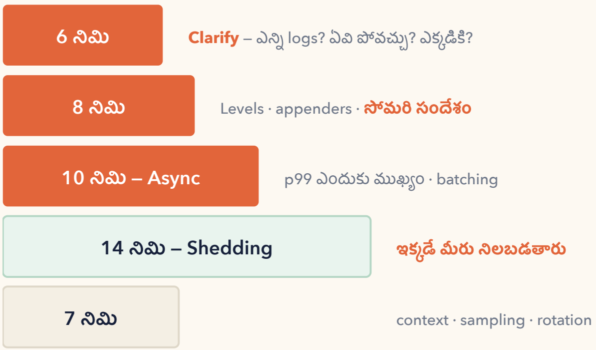
ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	స్థాయి + sink	మౌలిక అస్థిపంజరం
§4 (విరుపు)	సోమరి సందేశం, `isEnabled`	0 పంక్తులకి 2 లక్షల calls
§7 (విరుపు)	`LevelQueue` + `pump`	p99 1 ms → 21 ms
§8	`BufferedAppender`	50,000 syscalls → 100
§10 (విరుపు)	స్థాయి-తెలిసిన shedding	47% ERROR logs మాయం
§11	ఒక్కో స్థాయికి ఒక queue	O(n) shedding → O(1)
§11	`dropped` ఒక్కో స్థాయికీ	"4 లక్షల DEBUG" ≠ "3 ERROR"
§12	`child(fields)`	ఒక request యొక్క కథ ఒకచోట

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Levels మరియు appenders వేగంగా – queue విధానమే అసలు interview



ఏమి తప్పక చెప్పాలి

1. **ఆపేసిన log కూడా ఖరీదే** (§4) – "the argument is built before the level check runs." ఇది 30 సెకన్లు మరియు చాలా మంది దీన్ని ఎప్పుడూ ఆలోచించలేదు.
2. **Logging request path lo ఉండకూడదు** (§7) – మరియు **సగటు కాదు, p99** చెప్పండి.
3. **"ఏవి పోవచ్చు?" అని మీరే అడగండి** (§2, §10) – "logs can be lost" కాదు, "which logs can be lost".
4. **DROP_NEW మరియు DROP_OLD రెండూ ERROR లని మింగేస్తాయి** (§10) – **ఇదే ఈ interview యొక్క కేంద్రం**. 47% అనే సంఖ్య చెప్పకపోయినా, ఎందుకు అని వివరించండి: queue lo 70% DEBUG ఉంటే, క్రమం ప్రకారం వదిలేస్తే అదే నిష్పత్తిలో ERROR పోతుంది.
5. `child(fields)` **/ correlation id** (§12) – ఇది ఆచరణాత్మక అనుభవాన్ని చూపిస్తుంది.

ఏమి వదిలేయాలి

- `Record.format()` **రాయొద్దు** – "structured fields, rendered by the appender" ఒక వాక్యం.
- **File rotation, compression** – §16, అడిగితేనే.
- **Thread safety వివరాలు** – "the queue needs to be concurrent; in Java I'd use an ArrayBlockingQueue or a Disruptor" అని చెప్పి ముందుకి.
- `LevelQueue` **పూర్తిగా రాయొద్దు** – `push` lo shedding భాగం చాలు.

15. నోటితో చెప్పాల్సిన English script

"Three questions. How many log lines per second, and what does that look like during an incident rather than on a normal day? Where do they go – console, file, or over the network? And can logs be dropped – specifically, which ones?

I'll start with something small that surprises people. A logger has a level check, so a debug call in production costs nothing, right? It doesn't. When you write log.debug of 'request: ' plus describeRequest, the argument is evaluated before the call happens – the string concatenation, the JSON stringify, all of it – and only then does the logger check the level and throw it away. I measured two hundred thousand filtered debug calls: zero lines written, two hundred thousand invocations of the expensive function, about seventy milliseconds burned. And debug logging is densest in hot code, so that cost lands exactly where it hurts. The fix is to make the message lazy – accept a closure, or expose isEnabled, or take the message as separate fields and only join them after the level check. That took seventy milliseconds down to two.

Second: logging must not sit on the request path. Writing straight to a file is fine on average – about one and a half microseconds a line – but appenders aren't uniform. A disk flush, a log rotation, a network hiccup, and one call blocks for milliseconds. I simulated an appender that stalls twenty milliseconds every couple of hundred lines: p99 request latency went from one millisecond to twenty-one. The customer didn't ask for a log line and they paid for it. So the request thread does a queue push and nothing else, and a separate worker drains the queue into appenders in batches.

That queue is where the real design decision is, and it's the part I'd want to be judged on. Unbounded is not an option – during an incident log volume goes up by orders of magnitude, and an unbounded queue means the logging system kills the process. Logs shouldn't be able to take down the server they're describing. So it's bounded, and when it fills something has to go.

The two obvious policies are drop-newest and drop-oldest, and both are wrong in the same way. They shed by arrival order without looking at what they're shedding. If the queue is seventy percent debug lines, then dropping by position throws away errors in roughly that same proportion. I measured it on an incident-shaped workload: both policies lost about forty-seven percent of error lines. And notice when that happens – the queue only fills when the system is struggling, which is precisely the moment you're grepping for those errors.

So the queue should shed by level, not by position: drop the lowest-severity record present, and if the incoming record is itself the lowest, drop that. Same workload, a hundred percent of errors survive, and about ninety-five percent of debug lines are the ones sacrificed – which is exactly the trade you want. I'd keep one sub-queue per level so that's O(1) rather than scanning, and drain highest-severity first so a lagging consumer still emits the important lines. And I'd count drops per level, because four hundred thousand dropped debug lines is a health signal and three dropped errors is an alarm.

Last practical thing: a child logger that carries context – a request id, a user, a service name – so every line from one request is greppable as a unit without threading that id through every call."

16. Follow-ups – sampling, rotation, distributed tracing

Follow-up	జవాబు	మారే classes
"JSON గా రాయాలి"	`Record.format()` కి బదులు ఒక `JsonFormatter`	**+1 కొత్తది**
"ఒక్కో appender కి వేరే స్థాయి"	Appender కి సొంత `minLevel`	`write` lo ఒక filter
"File 100 MB దాటితే కొత్తది"	`RotatingAppender` – పరిమాణం/తేదీ ప్రకారం	**+1 కొత్తది**
"నడుస్తుండగా స్థాయి మార్చాలి"	`setLevel` ఇప్పటికే ఉంది	**0**
"ఎన్ని logs వదిలేశామో చూడాలి"	`queue.dropped` ఇప్పటికే ఉంది → Deep Dive 19 counters	**0 concepts**
"ఒకే error వెయ్యి సార్లు వస్తోంది"	కింద చూడండి	Sampling layer
"పది services మీద ఒక request"	కింద చూడండి	Tracing layer

Sampling – shedding కంటే తెలివైనది

> *"§11 lo నేను queue నిండాక వదిలేస్తున్నాను. కానీ ఒక సాధారణ నమూనా ఉంది: **ఒకే సందేశం వేల సార్లు**. ఒక DB కనెక్షన్ పోతే, ప్రతి request ఒకే ERROR రాస్తుంది.*
>
> ***అప్పుడు queue నింపడం అర్థరహితం** – ఆ వెయ్యి పంక్తుల్లో ఒకటి చదివితే చాలు. కాబట్టి queue కి ముందే ఒక **sampler**: "ఒకే సందేశం సెకనుకి గరిష్ఠంగా 10 సార్లు; మిగతావి లెక్కించి, చివర్లో 'ఇంకా 4,312 సార్లు' అని ఒక పంక్తి."*
>
> *ఇది **shedding కంటే మేలు** – shedding ఏది పోయిందో మర్చిపోతుంది; sampling **ఎన్ని జరిగాయో చెప్తుంది**. Go యొక్క `zap`, Rust యొక్క `tracing` – రెండింట్లోనూ ఇది ఉంది."*

Distributed tracing

> *"§12 యొక్క `child({ reqId })` ఒక్క సర్వర్ కి సరిపోతుంది. కానీ ఒక request పది services గుండా వెళితే?*
>
> ***అదే ఆలోచన, ఒక అడుగు ముందుకి:** ఆ id ని **services మధ్య పంపడం** – ఒక HTTP header గా. దానికి ఒక ప్రమాణం ఉంది: **W3C Trace Context** (`traceparent` header).*
>
> *అప్పుడు `traceId` అన్ని services logs lo కనిపిస్తుంది, మరియు ఒక్క query తో మొత్తం ప్రయాణం కనిపిస్తుంది.*
>
> ***మరియు గమనించండి – నా `fields` నిర్మాణం దీనికి సిద్ధంగా ఉంది.** `child({ traceId, spanId })` – కొత్త concept ఏమీ లేదు, ఒక కొత్త field మాత్రమే."*

---

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
తనిఖీ ఖర్చుకి ముందు ఉండాలి	సోమరి సందేశం (§5)	Assertions, feature flags, validation
సగటు కాదు, p99	2 ms సగటు, 21 ms p99 (§7)	ప్రతి latency సంభాషణ
అపరిమిత queue ఒక memory leak	§8	Deep Dive 07 §7 · producers/consumers
అన్ని items సమానం కావు	Level-aware shedding (§11)	Priority queues, QoS, load shedding
వదిలేసినదాన్ని కూడా లెక్కించు	`dropped` ఒక్క స్థాయికీ (§11)	Metrics, sampling, rate limits
Context ని మోయడం	`child(fields)` (§12)	Tracing, tenancy, auth
Optimisation ని reference తో పోల్చు	O(1) vs O(n) (§11)	Deep Dive 11 §8, 13 §5, 17 §12

ఒక చివరి ఆలోచన – ఈ PROBLEM యొక్క నిజమైన పాఠం

ఈ doc lo మూడు విరుపులూ ఒకే లక్షణాన్ని పంచుకుంటాయి: **అవి సాధారణ సమయంలో కనిపించవు.**

DEBUG ఆఫ్ ఉన్నప్పుడు ఖర్చు కనిపించదు – CPU గ్రాఫ్ కొంచెం ఎక్కువ ఉంటుంది, అంతే. Appender ఆగడం కనిపించదు – p50 బాగానే ఉంటుంది. Queue నిండటం కనిపించదు – సాధారణ రోజున అది ఎప్పుడూ నిండదు.

మూడూ ఒకేసారి కనిపిస్తాయి: ఒక ఘటన సమయంలో. CPU నిండినప్పుడు, disk నెమ్మదైనప్పుడు, errors పెరిగినప్పుడు – మరియు సరిగ్గా అప్పుడే మీరు logs మీద ఆధారపడతారు.

అందుకే logging framework ని "సాధారణ లోడ్" తో పరీక్షించడం అర్థరహితం. దాన్ని పరీక్షించాల్సినది అది **విఫలమయ్యే పరిస్థితిలోనే** – మరియు ఈ doc lo నేను చేసినది అదే: ఘటన సమయపు నిష్పత్తులతో, నెమ్మది consumer తో, ఆగే appender తో.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot** · **02 Cache** · **03 Rate Limiter** · **04 BookMyShow** · **05 Splitwise** · **06 Elevator** · **07 Pub-Sub** · **08 HashMap** · **09 Chess** · **10 Meeting Scheduler** · **11 Food Delivery** · **12 File System** · **13 Leaderboard** · **14 Text Editor** · **15 Library** · **16 Tic-Tac-Toe** · **17 Autocomplete** · **18 Snake & Ladder** · **19 Hit Counter** · **20 Vending Machine** · **21 ATM** · **22 Logging**.

Deep Dive 07 (Pub-Sub) ఈ doc కి జంట – అక్కడ queue ఒక decoupling సాధనం; ఇక్కడ queue ఒక ఎంపిక చేయాల్సిన చోటు.

Logging Framework – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసి తీసినదే ✅